

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6
По дисциплине: «ОМО»
Тема : “Реккурентные нейронные сети”

Выполнил:
Студент 3-го курса
Группы АС-66
Ярома А.И.
Проверил:
Крощенко А.А.

Цель: изучить рекуррентные нейронные сети.

Вариант 12

Задание:

1. По вариантам предыдущей лабораторной работы реализовать предложенный вариант рекуррентной нейронной сети. Сравнить полученные результаты с ЛР 5.

1	0.1	0.1	0.05	0.1	6	2	Элмана
---	-----	-----	------	-----	---	---	--------

```
import numpy as np
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
a = 0.1
```

```
b = 0.1
```

```
c = 0.05
```

```
d = 0.1
```

```
n_inputs = 64
```

```
n_hidden = 64
```

```
def target_function(x):
```

```
    return a * np.cos(b * x) + c * np.sin(d * x)
```

```
x = np.linspace(-100.0, 300.0, 4000)
```

```
y = target_function(x)
```

```
# Нормализация в [0, 1]
```

```
y_min = y.min()
```

```
y_max = y.max()
```

```
y_scaled = (y - y_min) / (y_max - y_min)
```

```
def create_dataset(series, n_inputs):
```

```
    X, T = [], []
```

```
    for i in range(len(series) - n_inputs):
```

```
X.append(series[i:i + n_inputs])    # окно длины n_inputs
T.append(series[i + n_inputs])      # следующий элемент
```

```
X = np.array(X, dtype=np.float32)
T = np.array(T, dtype=np.float32)
```

```
# (batch, seq_len, input_size)
X = X.reshape(-1, n_inputs, 1)
return X, T
```

```
X_all, T_all = create_dataset(y_scaled, n_inputs)
```

```
start_forecast_idx = np.where(x >= 100.0)[0][0]
end_forecast_idx = np.where(x <= 150.0)[0][-1]
n_forecast = end_forecast_idx - start_forecast_idx + 1
```

```
start_train = 0
end_train = start_forecast_idx - n_inputs # всё до  $x \approx 100$ , чтобы окно не залезало в
прогноз
```

```
X_train = X_all[start_train:end_train]
T_train = T_all[start_train:end_train]
```

```
X_train_t = torch.from_numpy(X_train) # (batch, seq_len, 1)
T_train_t = torch.from_numpy(T_train.reshape(-1, 1)) # (batch, 1)
```

```
class LSTMForecaster(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, num_layers=1):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True
        )
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        # out: (batch, seq_len, hidden_size)
```

```

out, (h_n, c_n) = self.lstm(x)
# берём скрытое состояние на последнем временном шаге
last_hidden = out[:, -1, :] # (batch, hidden_size)
y = self.fc(last_hidden)    # (batch, 1)
return y

```

```

model = LSTMForecaster(input_size=1, hidden_size=n_hidden, output_size=1)

```

```

criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.003)
epochs = 1000

```

```

train_losses = []

```

```

for epoch in range(epochs):
    model.train()
    optimizer.zero_grad()
    outputs = model(X_train_t)
    loss = criterion(outputs, T_train_t)
    loss.backward()
    optimizer.step()
    train_losses.append(loss.item())

```

```

model.eval()
with torch.no_grad():
    train_pred_scaled = model(X_train_t).numpy().flatten()

```

```

train_true_scaled = T_train

```

```

train_true = train_true_scaled * (y_max - y_min) + y_min
train_pred = train_pred_scaled * (y_max - y_min) + y_min

```

```

idx_train_targets = np.arange(start_train + n_inputs, end_train + n_inputs)
x_train_plot = x[idx_train_targets]

```

```

# -----
# Авторегрессионный прогноз вперед
# -----

```

```
start_window = y_scaled[start_forecast_idx - n_inputs:start_forecast_idx].astype(np.float32)
forecast_scaled = []
```

```
with torch.no_grad():
    window = start_window.copy()
    for k in range(n_forecast):
        x_seq = torch.from_numpy(window.reshape(1, n_inputs, 1))
        y_hat_scaled = model(x_seq).item()
        forecast_scaled.append(y_hat_scaled)
        # сдвигаем окно: убираем самое старое, добавляем прогноз
        window = np.roll(window, -1)
        window[-1] = y_hat_scaled
```

```
forecast_scaled = np.array(forecast_scaled, dtype=np.float32)
```

```
y_forecast = forecast_scaled * (y_max - y_min) + y_min
y_true_forecast = y[start_forecast_idx:end_forecast_idx + 1]
x_forecast = x[start_forecast_idx:end_forecast_idx + 1]
```

```
error_forecast = y_forecast - y_true_forecast
```

```
# -----
# Графики: обучение и прогноз
# -----
plt.figure(figsize=(16, 8))
plt.plot(x_train_plot, train_true, 'b-', label="Train true")
plt.plot(x_train_plot, train_pred, 'r--', label="Train prediction")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(16, 8))
plt.plot(x_forecast, y_true_forecast, 'g-', label="True")
plt.plot(x_forecast, y_forecast, 'r--', label="Forecast")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
# -----
# Графики: loss и ошибка прогноза
# -----
```

```
plt.figure(figsize=(12, 5))
plt.plot(train_losses)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(12, 5))
plt.plot(x_forecast, error_forecast)
plt.xlabel("x")
plt.ylabel("Forecast error")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
# -----
# Таблица и метрики
# -----
results_forecast = pd.DataFrame({
    "x": x_forecast,
    "True": y_true_forecast,
    "Forecast": y_forecast,
    "Error": error_forecast
})
```

```
print(results_forecast.head(10).to_string(index=False))
print("\nMAE:", np.mean(np.abs(error_forecast)))
print("Max error:", np.max(np.abs(error_forecast)))
```

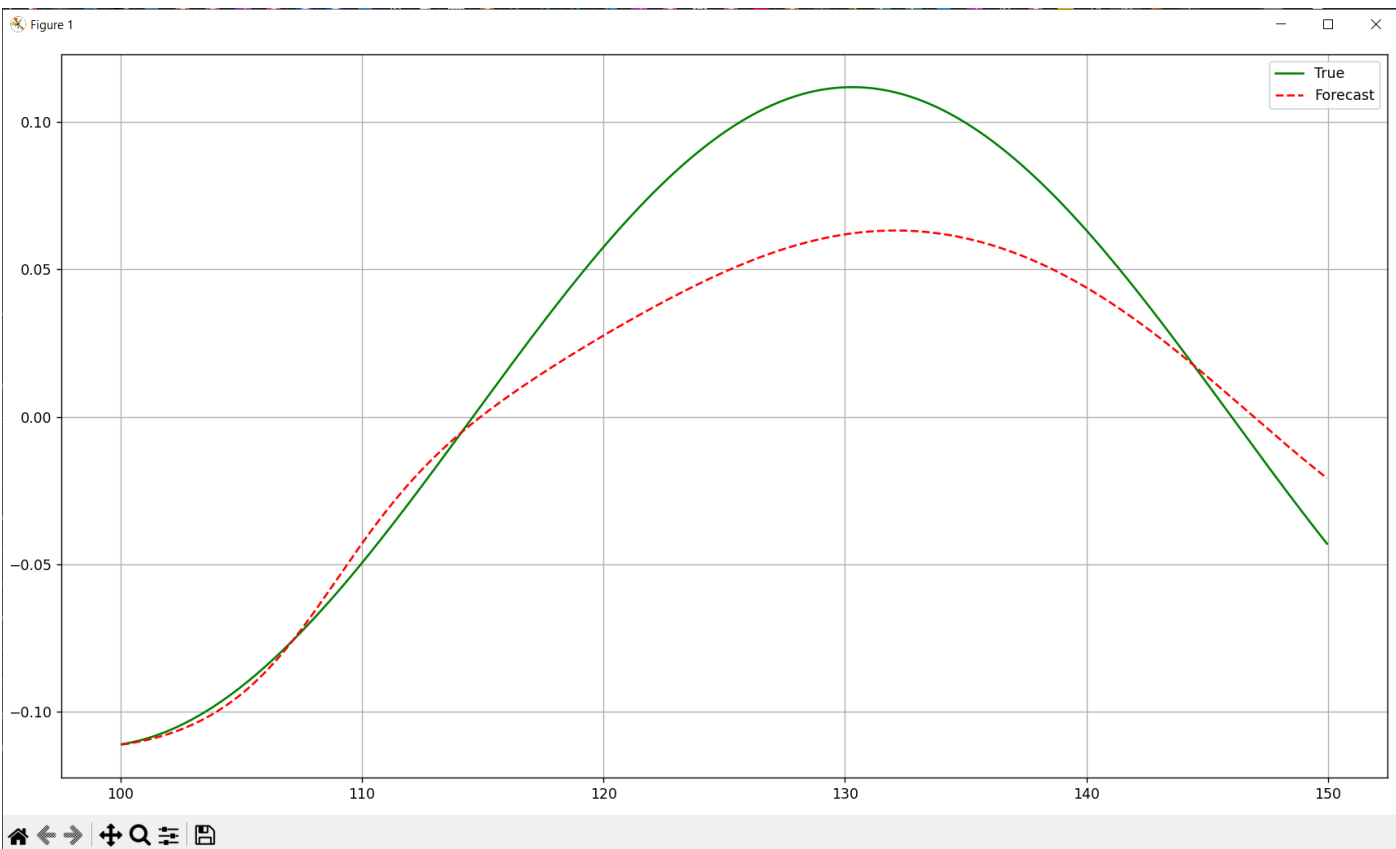
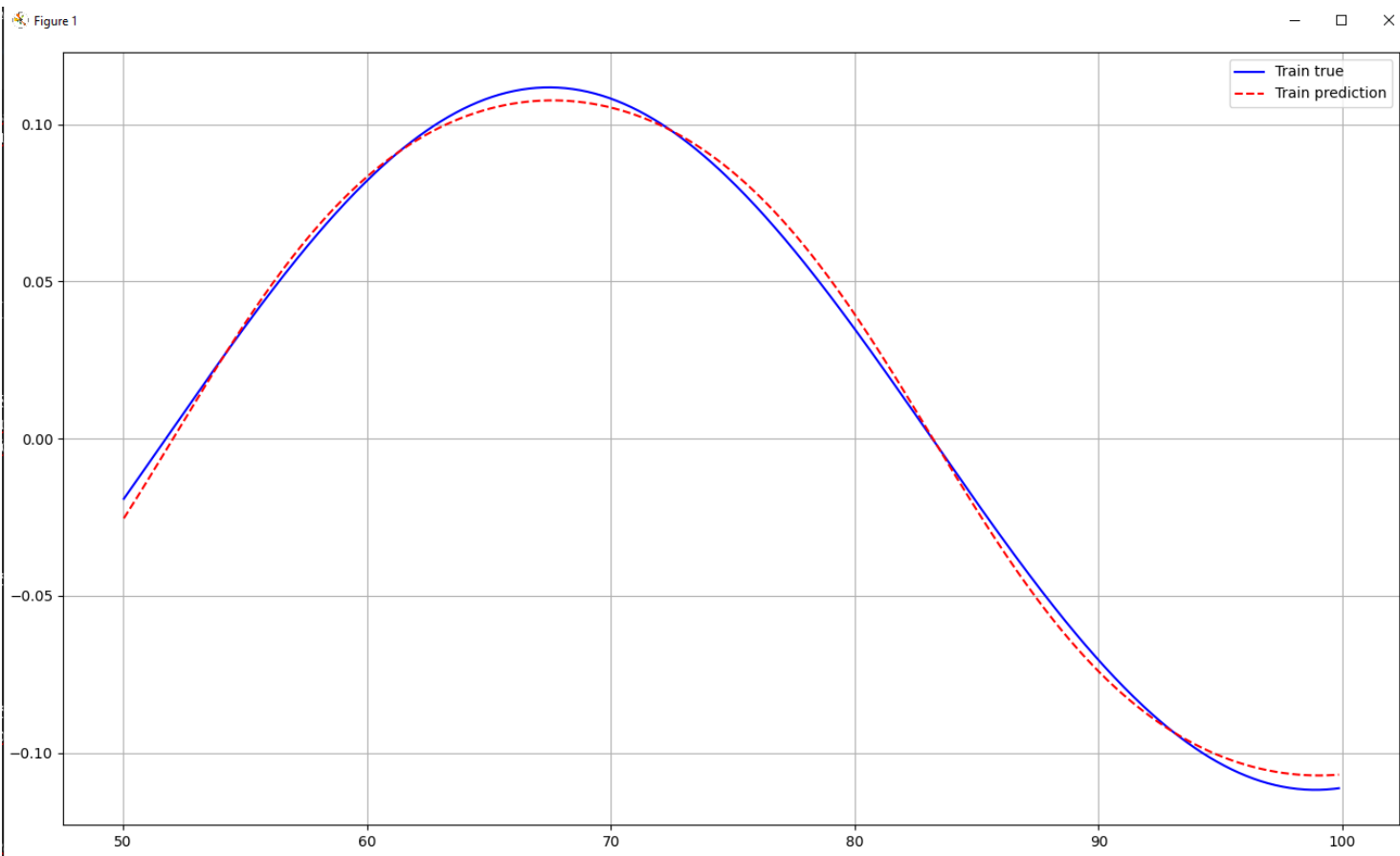


Figure 1

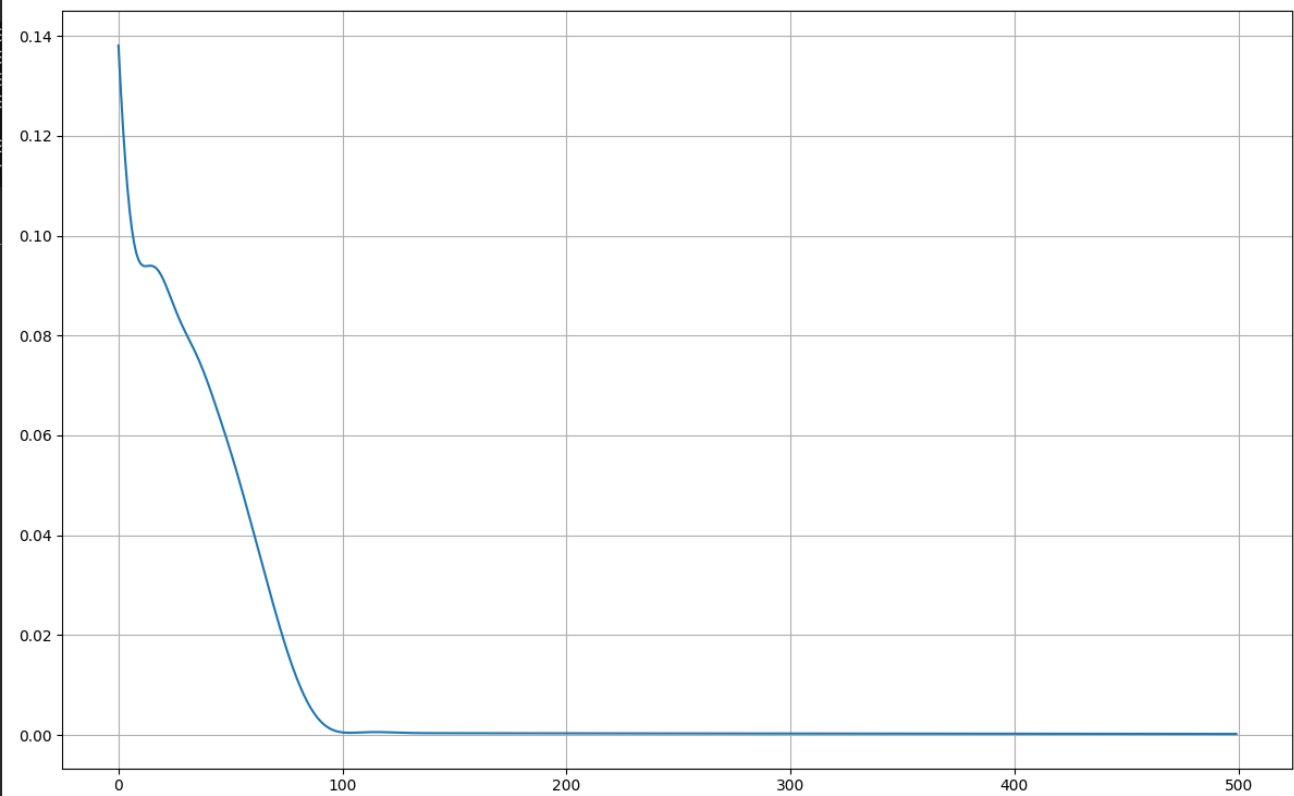
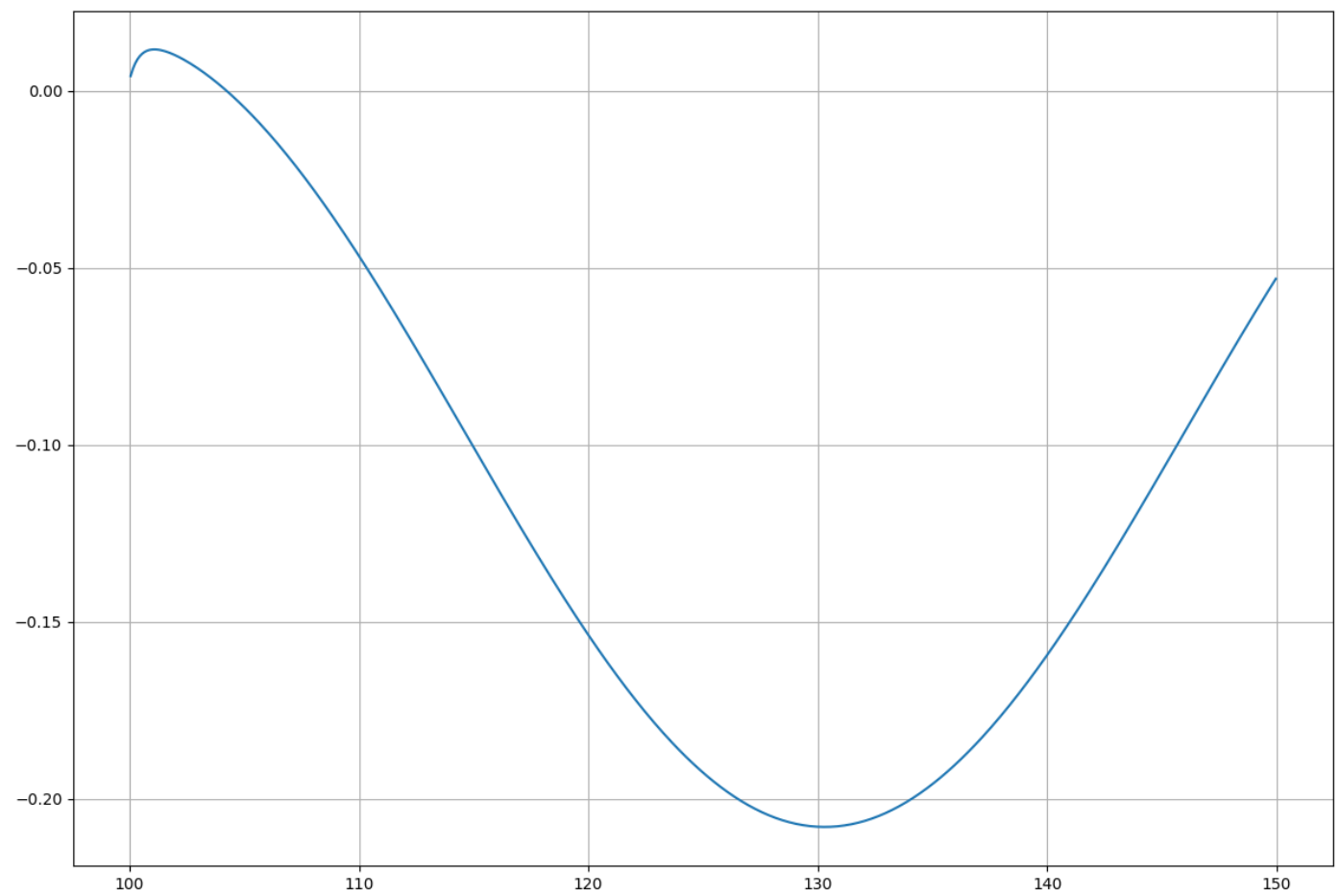


Figure 1



Вывод: построили, обучили и оценили многослойный перцептрон (MLP) для решения задачи классификации. На практике сравнили работу нескольких алгоритмов классификации.